

به نام خدا



هوش مصنوعی
نیم سال دوم ۰۴-۰۵
استاد: دکتر نصیری مفخم
شماره گروه : ۱۸
اعضای گروه:

فرنوش پورشبان (۴۰۲۴۰۲۳۰۰۷)
فرزانه سلیمی (۴۰۱۴۰۱۳۰۵۹)
یگانه رستگاری (۴۰۱۴۰۱۳۰۴۰)

دانشکده ریاضی و آمار

پروژه سری سوم

«جستجوی خصمانه برای بازی اتلو (Othello)»

پیش گفتار:

در این پروژه قصد داریم تا با استفاده از مفهوم جستجوی خصمانه (Adversarial Search) به طراحی و پیاده سازی عامل های هوشمند برای بازی اتلو (Othello) پردازیم. بازی اتلو یک بازی دونفره رقابتی با اطلاعات کامل است که در آن هر بازیکن باید با در نظر گرفتن حرکات احتمالی حریف، بهترین تصمیم ممکن را برای وضعیت فعلی در صفحه و حرکت بعدی خود را انتخاب کند. به همین دلیل تصمیم گیری در چنین بازی هایی تنها بر اساس وضعیت فعلی کافی نیست و لازم است پیامدهای هر حرکت در چند مرحله ی بعد نیز مورد بررسی قرار گیرد.

در این پروژه برای مدل سازی این فرآیند تصمیم گیری از الگوریتم های Minimax و Alpha-Beta Pruning استفاده شده است. الگوریتم Minimax با فرض بازی عقلانی هر دو بازیکن، فضای حالات بازی را جستجو کرده و حرکتی را انتخاب می کند که بهترین نتیجه را برای عامل به همراه داشته باشد. در کنار آن، الگوریتم Alpha-Beta Pruning به عنوان نسخه ای بهینه تر از Minimax پیاده سازی شده است که با حذف شاخه هایی از درخت جستجو که تأثیری در نتیجه نهایی ندارند، باعث کاهش تعداد محاسبات و افزایش کارایی می شود. برای ارزیابی عملکرد این

عامل‌ها، آن‌ها را در برابر دو نوع عامل ساده‌تر یعنی Random که حرکات را به صورت تصادفی انتخاب می‌کند و Greedy که در هر مرحله بهترین سود آنی را در نظر می‌گیرد قرار دادیم.

در ادامه ابتدا منطق مسئله و الگوریتم‌های مورد استفاده را توضیح داده و سپس ساختار کدها و پیاده‌سازی هر کلاس به صورت جداگانه بررسی خواهد شد.

لینک گیت‌هاب پروژه:

<https://github.com/Farnoush2005/Adversarial-Othello-AI>

تحلیل عمیق مکانیزم عملکرد الگوریتم‌ها:

برای درک بهتر نتایج آزمایش‌ها، لازم است نحوه‌ی تصمیم‌گیری و چگونگی پیمایش درخت بازی توسط دو الگوریتم Minimax و Alpha-Beta Pruning را دقیق‌تر بررسی کنیم.

۱. مکانیزم عملکرد الگوریتم Minimax:

الگوریتم مینی‌ماکس یک روش جستجوی بازگشتی و کامل است که در آن فرض بر این است که حریف ما نیز به صورت کاملاً هوشمندانه بازی می‌کند و در هر نوبت، حرکتی را انتخاب خواهد کرد که کمترین سود را برای ما و بیشترین سود را برای خودش داشته باشد؛ به همین دلیل، این الگوریتم سعی می‌کند تا چند حرکت جلوتر از بازی را پیش‌بینی و شبیه‌سازی کند و بهترین تصمیم ممکن را بر اساس آن بگیرد.

در شبیه‌سازی آینده، فرآیند عملکرد این الگوریتم از وضعیت فعلی صفحه آغاز می‌شود و تمام حرکات قانونی ممکن را به ترتیب بررسی می‌کند. این بررسی تا رسیدن به عمق تعیین‌شده ادامه پیدا می‌کند و در نهایت، در گره‌های برگ تابع

ارزیابی `evaluate_board` روی هر وضعیت اجرا می‌شود تا برای آن یک امتیاز عددی محاسبه شود. سپس در یک حرکت رو به بالا، این امتیازها از پایین درخت به سمت ریشه برگردانده می‌شوند.

در لایه‌های Max، الگوریتم فرض می‌کند که ما بهترین حرکت را انتخاب می‌کنیم، بنابراین از بین امتیازهای فرزندان، مقدار بزرگ‌تر را به بالا منتقل می‌کند. در مقابل، در لایه‌های Min فرض بر این است که حریف می‌خواهد نتیجه را برای ما بدتر کند، پس مقدار کوچک‌تر انتخاب می‌شود. با وجود این که Minimax از نظر منطق تصمیم‌گیری دقیق و کامل است، یک ضعف مهم دارد. این الگوریتم بدون هیچ‌گونه بهینه‌سازی، تمام شاخه‌های درخت را بررسی می‌کند.

بنابراین اگر در هر مرحله ۱۰ حرکت مجاز وجود داشته باشد و عمق جستجو ۴ در نظر گرفته شود، تعداد حالات بررسی شده به $10^4 = 10,000$ می‌رسد. همین موضوع باعث می‌شود با افزایش عمق، زمان اجرای الگوریتم به صورت نمایی افزایش پیدا کند؛ بنابراین کارایی آن به شدت کاهش می‌یابد.

۲. مکانیزم عملکرد الگوریتم Alpha-Beta Pruning:

الگوریتم آلفا-بتا از نظر خروجی نهایی و کیفیت حرکت، دقیقاً همان نتیجه Minimax را تولید می‌کند، اما مسیر جستجوی آن هوشمندانه‌تر و کم‌هزینه‌تر است. این الگوریتم دو مقدار کمکی به نام‌های آلفا (α) و بتا (β) را در طول جستجو نگه می‌دارد.

آلفا نشان‌دهنده بهترین امتیازی است که بازیکن ماکزیمم‌کننده تا آن لحظه به دست آورده یا تضمین کرده است و مقدار اولیه آن برابر با $-\infty$ در نظر گرفته می‌شود. در مقابل، بتا بهترین امتیازی را نشان می‌دهد که بازیکن مینیمم‌کننده تاکنون برای خود حفظ کرده است و مقدار اولیه آن $+\infty$ است.

منطق اصلی این الگوریتم در هرس کردن شاخه‌ها است. هر زمان که در طول جستجو شرط $\beta \leq \alpha$ برقرار شود، الگوریتم بلافاصله ادامه بررسی آن شاخه را متوقف می‌کند (break)؛ زیرا اگر شرط $\beta \leq \alpha$ برقرار شود، یعنی حریف در این شاخه گزینه‌ای دارد که امتیاز را به حدی پایین می‌آورد (β) که از امتیاز تضمین‌شده ما در یک شاخه دیگر (α) کمتر است.

از آنجا که ما (به عنوان بازیکن ماکزیم کننده) هرگز داوطلبانه وارد شاخه‌ای با امتیاز کمتر نمی‌شویم، بررسی ادامه حرکات آن فرزند عملاً «اتلاف وقت» است و هرس کردن آن هیچ تأثیری در نتیجه نهایی ندارد. بنابراین این مکانیزم به شکلی باعث می‌شود بدون فدا کردن دقت، بخش بزرگی از درخت جستجو حذف شود.

تابع ارزیابی:

تابع ارزیابی (Evaluation Function) طراحی شده در این پروژه، یک ترکیب خطی از سه معیار ابتکاری (Heuristic) در بازی اتلو است. در این روش، مقدار خام هر Heuristic نسبت به حداکثر مقدار ممکن آن در طول بازی محاسبه و سپس با یک نگاشت خطی به بازه‌ی هدف منتقل می‌شود تا وزن‌دهی نهایی به درستی صورت گیرد. برای اینکه خروجی هر معیار در یک بازه‌ی یکسان و قابل مقایسه قرار بگیرد و در نهایت مقادیر در بازه‌ی 100- تا 100+ نرمال‌سازی شوند، از فرمول‌های درصدی استفاده می‌کنیم:

۱. توازن مهره‌ها (Coin Parity): نسبت تعداد مهره‌های ما به مهره‌های حریف را محاسبه می‌کند. (وزن: ۱)

۲. تحرک پذیری (Mobility): تعداد حرکت‌های مجاز ما نسبت به حریف را می‌سنجد. هرچه حرکات بیشتری داشته باشیم، دست ما برای بازی بازتر است و حریف محدودتر می‌شود. (وزن: ۲.۵)

۳. تصرف گوشه‌ها (Corners Occupancy): در بازی اتلو، مهره‌هایی که در ۴ گوشه صفحه قرار می‌گیرند هرگز نمی‌توانند توسط حریف محاصره و برگردانده شوند. بنابراین گرفتن گوشه‌ها مهم‌ترین و بهترین استراتژی است. (وزن: ۱۵)

امتیاز نهایی با فرمول زیر محاسبه می‌شود:

$$Score = Core Parity + (Mobility * 2.5) + (Corners Occupancy * 15)$$

پیاده سازی الگوریتم‌ها:

۱. ایده پیاده سازی کلاس *MinimaxAgent*:

این کلاس، عامل هوشمندی را پیاده‌سازی می‌کند که تمام حرکات ممکن را تا یک عمق مشخص بررسی کرده و بر اساس نتیجه نهایی، بهترین حرکت را انتخاب می‌کند.

❖ متد `__init__`: عمق جستجو (Depth) را برای محدود کردن درخت بازی دریافت و ذخیره می‌کند.

❖ متد `evaluate_board`: این متد همان تابع ارزیابی است (در بخش ۳ مفصل توضیح دادیم). اگر بازی تمام شده باشد، برای برد امتیاز بسیار بالا مثلاً (۹۹۹۹۹۹) و برای باخت امتیاز بسیار پایین (۹۹۹۹۹۹-) برمی‌گرداند.

❖ متد `run_minimax`: این متد کار اصلی الگوریتم را انجام می‌دهد که به صورت بازگشتی کار می‌کند به این شکل که چندین پارامتر داریم:

- شرط توقف (Base Case): اگر به عمق صفر برسیم یا بازی تمام شود، تابع ارزیابی فراخوانی شده و امتیاز وضعیت فعلی برگردانده می‌شود.

- قانون پاس دادن: با استفاده از `get_valid_moves` حرکات مجاز بررسی می‌شود. اگر بازیکنی حرکتی نداشته باشد، نوبتش پرش (Pass) می‌شود.

- بازیکن ماکزیمم‌کننده (`is_my_turn == True`): تمام حرکات مجاز شبیه‌سازی می‌شوند (`game.copy` و `make_move`). هدف پیدا کردن بیشترین امتیاز (`best_score = float('-inf')`) است.

- بازیکن مینیمم‌کننده (`is_my_turn == False`): نشان‌دهنده نوبت حریف است. هدف پیدا کردن کمترین امتیاز (`best_score = float('inf')`) است تا بدترین سناریو برای ما در نظر گرفته شود.

❖ متد `choose_move`: با فراخوانی اولیه `run_minimax` بهترین حرکت را برای اعمال روی صفحه اصلی بازی انتخاب می‌کند.

❖ تحلیل و توضیح کد:

این کلاس به صورت اختصاصی مسئولیت پیاده سازی منطق تصمیم گیری بر پایه الگوریتم کلاسیک مینی ماکس را بر عهده دارد و به agent ما این توانایی را می دهد تا حرکات خود را بر اساس این استراتژی انتخاب کند.

۱. متد مقداردهی اولیه (__init__):

```
class MinimaxAgent:  
    def __init__(self, depth=4):  
        self.depth = depth
```

این متد سازنده، عمق مجاز برای پیمایش در درخت بازی را به عنوان ورودی دریافت کرده و آن را در متغیر self.depth ذخیره می کند؛ در پیاده سازی فعلی، مقدار پیش فرض این عمق روی ۴ تنظیم شده است تا تعادلی میان کیفیت تصمیم گیری و زمان محاسبات برقرار باشد.

۲. متد ارزیابی صفحه (evaluate_board):

این تابع کیفیت یک وضعیت مشخص از صفحه بازی را برای بازیکن ما می سنجد و خروجی آن یک عدد مشخص است که برتری یا ضعف وضعیت فعلی را نشان می دهد.

```
def evaluate_board(self, game, my_color):  
    enemy_color = -my_color
```

ابتدا رنگ مهره حریف (enemy_color) بر اساس معکوس کردن رنگ مهره ما محاسبه می شود (یعنی اگر ما ۱ باشیم حریف ۱- است).

```
# چک کردن اینکه بازی تموم شده یا نه
if game.game_over():
    black_score, white_score = game.score()
    my_score = black_score if my_color == 1 else white_score
    enemy_score = white_score if my_color == 1 else black_score

    # دادن به عدد خیلی بزرگ برای برد و خیلی کوچک برای باخت
    if my_score > enemy_score:
        return 999999
    elif my_score < enemy_score:
        return -999999
    return 0
```

بررسی می‌شود که آیا بازی تمام شده است یا خیر (`game.game_over()`). اگر تمام شده باشد، امتیاز نهایی مهره‌های سیاه و سفید استخراج می‌شود. با یک شرط ساده، امتیاز ما و حریف تفکیک شده و در صورت برد ما، یک عدد بسیار بزرگ مثبت (وضعیت ایده‌آل) و در صورت باخت، یک عدد بسیار کوچک منفی (وضعیت بحرانی) و در صورت مساوی عدد صفر بازگردانده می‌شود.

```
# معیار اول: اختلاف مهره‌ها
black_score, white_score = game.score()
my_score = black_score if my_color == 1 else white_score
enemy_score = white_score if my_color == 1 else black_score
coin_diff = 100 * (my_score - enemy_score) / (my_score + enemy_score + 1)
```

معیار اول (تعداد مهره‌ها):

اگر بازی هنوز به پایان نرسیده باشد، این معیار تعداد مهره‌های فعلی روی صفحه را می‌شمارد. طبق فرمولی که در بالا پیاده‌سازی کردیم، ابتدا اختلاف تعداد مهره‌های ما و حریف محاسبه می‌شود؛ سپس برای اینکه خروجی را بین بازه -100 تا $+100$ نرمال‌سازی کنیم، این اختلاف را بر مجموع مهره‌های موجود تقسیم می‌کنیم. نکته مهم در اینجا، اضافه کردن عدد ۱ در مخرج است که برای جلوگیری از خطای احتمالی «تقسیم بر صفر» در مراحل آغازین بازی (که ممکن است تعداد مهره‌ها هنوز بسیار کم باشد) در نظر گرفته شده است.

```
# معیار دوم: تعداد حرکات مجاز ما نسبت به حریف (تحریک پذیری)
my_moves = len(game.get_valid_moves(my_color))
enemy_moves = len(game.get_valid_moves(enemy_color))
mobility_score = 0
if (my_moves + enemy_moves) > 0:
    mobility_score = 100 * (my_moves - enemy_moves) / (my_moves + enemy_moves)
```

معیار دوم (تحریک پذیری):

در این بخش، تعداد حرکات مجاز برای ما و حریف استخراج می‌شود. منطق کار به این صورت است که اگر مجموع حرکات مجاز صفر نباشد، ما نسبت اختلاف حرکات مجاز به کل حرکات ممکن را محاسبه می‌کنیم؛ این شاخص به ما دید خوبی می‌دهد که در وضعیت فعلی، چقدر «کنترل بازی» و حق انتخاب در دست داریم و تا چه حد می‌توانیم فضای مانور حریف را محدود کنیم.

```
# معیار سوم: گرفتن گوشه‌های صفحه که امتیاز بالایی دارد
corners = [(0, 0), (0, game.size - 1), (game.size - 1, 0), (game.size - 1, game.size - 1)]
my_corners = 0
enemy_corners = 0

# با به حلقه ساده چک میکنیم گوشه‌ها دست کیه
for r, c in corners:
    if game.board[r][c] == my_color:
        my_corners += 1
    elif game.board[r][c] == enemy_color:
        enemy_corners += 1
```

معیار سوم (تصاحب گوشه‌ها):

از آنجایی که گوشه‌ها در بازی اتلو اهمیت استراتژیک بالایی دارند و معمولاً غیرقابل بازپس‌گیری هستند، مختصات ۴ گوشه صفحه ۶×۶ را از پیش تعریف کردیم. در ادامه، با استفاده از یک حلقه ساده، وضعیت این خانه‌ها را پیمایش می‌کنیم تا مشخص شود در لحظه فعلی، چند گوشه توسط مهره‌های ما اشغال شده و حریف چه سهمی از این نقاط کلیدی را در اختیار دارد. این وزن‌دهی به گوشه‌ها کمک می‌کند تا عامل ما در حین جستجو، اولویت بیشتری برای تصاحب یا حفظ خانه‌های استراتژیک قائل شود.


```

corner_score = 0
if (my_corners + enemy_corners) > 0:
    corner_score = 100 * (my_corners - enemy_corners) / (my_corners + enemy_corners)

# جمع بندی امتیازات با وزنهای مشخص شده
return coin_diff + (2.5 * mobility_score) + (15.0 * corner_score)

```

امتیاز مربوط به گوشه‌ها هم مانند سایر معیارها به بازه ۱۰۰- تا ۱۰۰+ نرمال‌سازی می‌شود تا بتوان آن را با بقیه مؤلفه‌های ارزیابی در یک مقیاس مشترک مقایسه کرد. در نهایت، همه امتیازها با وزن‌های از پیش تعیین‌شده با هم ترکیب می‌شوند؛ به این شکل که تحرک‌پذیری با ضریب ۲.۵ و گوشه‌ها با ضریب سنگین‌تر ۱۵ در مقدار نهایی لحاظ می‌شوند. حاصل این جمع وزن‌دار، ارزش نهایی آن وضعیت از صفحه را به عامل می‌دهد و مشخص می‌کند که این حالت تا چه اندازه به نفع بازیکن ما است.

۳. تابع بازگشتی مینی‌مکس (run_minimax):

```

def run_minimax(self, game, current_depth, is_my_turn, my_color):
    # اگر به انتهای عمق رسیدیم یا بازی تمام شد، امتیاز صفحه رو برمی‌گردانیم
    if current_depth == 0 or game.game_over():
        return self.evaluate_board(game, my_color), None

```

شرط توقف:

اگر عمق جستجو به صفر برسد یا وضعیت بازی به پایان رسیده باشد، تابع بازگشتی متوقف شده و ارزش صفحه را به همراه None (به نشانه عدم وجود حرکت بعدی) برمی‌گرداند.

```

# پیدا کردن اینکه الان نوبت کیه
current_player = my_color if is_my_turn else -my_color
valid_moves = game.get_valid_moves(current_player)

```

در این بخش مشخص می‌شود که در این لایه از درخت نوبت بازیکن ما کزیم‌کننده (خودمان) است یا مینیمم‌کننده (حریف). سپس حرکات قانونی آن بازیکن استخراج می‌شود.

```
# قانون پاس دادن تو اتلو: اگر بازیکنی حرکت نداشت نوبتش میبهره
if len(valid_moves) == 0:
    score, _ = self.run_minimax(game, current_depth - 1, not is_my_turn, my_color)
    return score, None
```

قانون پاس:

در پیاده‌سازی منطق بازی، باید شرایطی را در نظر بگیریم که بازیکن در وضعیت فعلی امکان هیچ حرکت مجاز و قانونی ندارد. طبق قوانین بازی اتلو، در چنین حالتی نوبت بازیکن پاس داده می‌شود. برای مدیریت این وضعیت در الگوریتم، نوبت بازی را تغییر می‌دهیم (not is_my_turn) و برای اینکه عمق جستجو در درخت بازگشتی به درستی مدیریت شود، عمق فعلی را یک واحد کاهش می‌دهیم تا الگوریتم بدون توقف، وضعیت بعدی بازی را با نوبت حریف (یا برعکس) در سطوح پایین‌تر ارزیابی کند.

```
if is_my_turn:
    best_score = float('-inf')
    best_move = None
    for move in valid_moves:
        new_game = game.copy()
        new_game.make_move(current_player, move[0], move[1])
        score, _ = self.run_minimax(new_game, current_depth - 1, False, my_color)

        # پیدا کردن ماکزیمم امتیاز
        if score > best_score:
            best_score = score
            best_move = move
    return best_score, best_move
```

بخش ماکزیمم‌کننده:

اگر نوبت ما باشد، امتیاز اولیه منفی بی‌نهایت فرض می‌شود. تک‌تک حرکات مجاز در حلقه بررسی می‌شوند؛ یک نسخه کپی از بازی ایجاد شده، حرکت روی آن اعمال می‌شود و تابع به صورت بازگشتی برای لایه بعدی (که نوبت حریف است: False) صدا زده می‌شود. در نهایت، حرکتی که بیشترین امتیاز را تولید کند ذخیره و برگشت داده می‌شود.

```

else:
    best_score = float('inf')
    best_move = None
    for move in valid_moves:
        new_game = game.copy()
        new_game.make_move(current_player, move[0], move[1])
        score, _ = self.run_minimax(new_game, current_depth - 1, True, my_color)

        # پیدا کردن مینیمم امتیاز
        if score < best_score:
            best_score = score
            best_move = move
    return best_score, best_move

```

بخش مینیمم کننده:

در این بخش، اگر نوبت حریف باشد، منطق کاملاً برعکس حالت ماکزیمم کننده عمل می کند. در این حالت، مقدار اولیه امتیاز را مثبت بی نهایت در نظر می گیریم تا الگوریتم بتواند به تدریج مقادیر کوچک تر را جایگزین آن کند و در نهایت، حرکتی را پیدا کند که امتیاز ما را به کمترین مقدار ممکن برساند؛ زیرا فرض بر این است که حریف یک بازیکن هوشمند است و بهترین تصمیم ممکن را برای کاهش ارزش وضعیت ما انتخاب می کند.

```

def choose_move(self, game, player):
    # شروع حرکت با فراخوانی تابع مینی مکس
    score, move = self.run_minimax(game, self.depth, True, player)
    return move

```

این متد به عنوان رابط اصلی عمل کرده، فرآیند مینی مکس را با عمق اولیه و در حالی که نوبت ماست (True) آغاز می کند و در نهایت فقط بهترین حرکت منتخب را به محیط بازی تحویل می دهد.

۲. ایده ی پیاده سازی کلاس Alpha-BetaAgent:

این کلاس نسخه بهینه شده ی Minimax است که با استفاده از هرس کردن (Pruning)، از بررسی شاخه هایی که تأثیری در نتیجه نهایی ندارند، جلوگیری کند.

❖ توابع __init__ و evaluate_board مشابه تعریف شده اند Minimax هستند تا شرایط مقایسه عادلانه باشد.

❖ متد run_alphabeta:

عملکرد کلی آن شبیه به متد run_minimax است. تنها تفاوت اضافه شدن دو متغیر α (بهترین

امتیاز تضمین شده برای ما) و β (بهترین امتیاز تضمین شده برای حریف) است:

- هرس آلفا-بتا: در طول شبیه سازی حرکات، متغیرهای آلفا و بتا آپدیت می شوند. اگر در هر لحظه

$\alpha \geq \beta$ شود، حلقه با دستور break متوقف می شود. این یعنی ادامه جستجو در این

شاخه بی فایده است زیرا حریف یا ما هرگز اجازه رسیدن به این وضعیت را نخواهیم داد.

❖ تحلیل و توضیح کد:

از آنجا که ساختار تابع ارزیابی و توابع کلی این کلاس دقیقاً با Minimax یکسان است، تمرکز اصلی گزارش

را بر روی متد run_alphabeta و منطق هرس کردن قرار می دهیم.

```
def run_alphabeta(self, game, current_depth, alpha, beta, is_my_turn, my_color):
```

ورودی های این متد علاوه بر پارامترهای Minimax، شامل دو متغیر حیاتی α (بهترین امتیاز تضمین

شده برای بازیکن ماکزیم کننده) و β (بهترین امتیاز تضمین شده برای بازیکن مینیمم کننده) است. شرط

پایه، تعیین بازیکن و قانون پاس در این کلاس کاملاً مشابه مینی ماکس عمل می کنند.

```

# رسیدن به انتهای عمق تعیین شده یا تموم شدن بازی
if current_depth == 0 or game.game_over():
    return self.evaluate_board(game, my_color), None

current_player = my_color if is_my_turn else -my_color
valid_moves = game.get_valid_moves(current_player)

# قانون پاس دادن: اگر بازیکنی حرکت مجازی نداشت، نوبتش می‌پوره
if len(valid_moves) == 0:
    score, _ = self.run_alphabeta(game, current_depth - 1, alpha, beta, not is_my_turn, my_color)
    return score, None

if is_my_turn:
    best_score = float('-inf')
    best_move = None
    for move in valid_moves:
        new_game = game.copy()
        new_game.make_move(current_player, move[0], move[1])

        score, _ = self.run_alphabeta(new_game, current_depth - 1, alpha, beta, False, my_color)

        if score > best_score:
            best_score = score
            best_move = move

        alpha = max(alpha, best_score)
        if beta <= alpha:
            break # هرس کردن شاخه
    return best_score, best_move

```

هرس در بخش ماکزیمم:

پس از اینکه امتیاز حرکت فعلی از لایه‌های پایین‌تر درخت دریافت شد، اگر این امتیاز از $best_score$ بزرگتر باشد، جایگزین می‌شود. مقدار $alpha$ آپدیت می‌شود ($\max(alpha, best_score)$).

در خط بعدی شرط هرس بررسی می‌شود. اگر مقدار $beta$ کوچکتر یا مساوی $alpha$ شود، یعنی حریف در لایه‌های بالاتر حرکتی دارد که اجازه نمی‌دهد بازی به این شاخه برسد؛ بنابراین با دستور `break` حلقه را متوقف کرده و از بررسی بقیه حرکات این شاخه صرف نظر می‌کنیم (هرس می‌کند).

```

else:
    best_score = float('inf')
    best_move = None
    for move in valid_moves:
        new_game = game.copy()
        new_game.make_move(current_player, move[0], move[1])

        score, _ = self.run_alphabeta(new_game, current_depth - 1, alpha, beta, True, my_color)

        if score < best_score:
            best_score = score
            best_move = move

        beta = min(beta, best_score)
        if beta <= alpha:
            break # هرس کرین شاخه
    return best_score, best_move

```

هرس در بخش مینیمم:

در نوبت حریف، پس از ارزیابی هر حرکت، مقدار β به روزرسانی می‌شود ($\min(\beta, \text{best_score})$). مجدداً شرط $\alpha \geq \beta$ بررسی می‌شود؛ اگر برقرار باشد، یعنی ما در لایه‌های بالاتر حرکتی داریم که مانع انتخاب این مسیر توسط ما می‌شود، پس حریف بیهوده در حال بهینه‌سازی این شاخه است و ادامه حلقه هرس (break) می‌شود.

```

def choose_move(self, game, player):
    # شروع الگوریتم با مقادیر اولیه آلفا و بتا
    score, move = self.run_alphabeta(game, self.depth, float('-inf'), float('inf'), True, player)
    return move

```

این تابع الگوریتم Alpha-Beta را با مقادیر اولیه منفی بی‌نهایت برای آلفا و مثبت بی‌نهایت برای بتا استارت می‌زند.

۳. ایده فایل اجرایی `main.py`:

این فایل وظیفه اجرای مسابقات (Round-Robin Tournament) و ارزیابی عملکرد عامل‌ها را بر عهده دارد.

❖ تابع `test_agents`: دو عامل را دریافت کرده و یک حلقه به تعداد بازی‌های مشخص شده (مثلاً ۲۰ بازی) اجرا می‌کند.

❖ تعویض رنگ مهره‌ها: برای رعایت عدالت، در بازی‌های زوج عامل اول با مهره سیاه (شروع‌کننده) و در بازی‌های فرد با مهره سفید بازی می‌کند ($i \% 2 == 0$).

❖ ثبت نتایج: پس از پایان هر بازی، خروجی تابع `score` بررسی شده و تعداد بردها و تساوی‌ها شمرده شده و در نهایت درصد برد (Win Rate) چاپ می‌شود.

❖ تحلیل و توضیح فایل اجرای آزمایش‌ها (`main.py`):

این فایل ساختار لازم برای برگزاری مسابقات خودکار و استخراج جداول نرخ برد را فراهم می‌کند.

```
from game.othello import Othello, BLACK, WHITE
from tournament import play_game
from agents.random_agent import RandomAgent
from agents.greedy_agent import GreedyAgent
from agents.minimax_agent import MinimaxAgent
from agents.alphabeta_agent import AlphaBetaAgent
```

ابتدا در کد `main` کلاس‌ها را `import` می‌کنیم.

```
def test_agents(agent1, agent2, num_games=20):
    # متغیرها برای نگه داشتن تعداد برد و باخت و تساوی
    agent1_wins = 0
    agent2_wins = 0
    draws = 0
```

تابع سه ورودی (عامل اول، عامل دوم و تعداد کل بازی‌ها) را دریافت کرده و شمارنده‌های برد و مساوی را در ابتدا صفر می‌کند.

```
for i in range(num_games):
    # برای اینکه عادلانه باشد، جای مهره سیاه و سفید رو تو بازی‌های زوج و فرد عوض می‌کنیم
    if i % 2 == 0:
        score = play_game(agent1, agent2)
        score1 = score[0]
        score2 = score[1]
```

برقراری عدالت در رنگ‌ها: یک حلقه به تعداد کل بازی‌ها اجرا می‌شود. اگر شماره بازی زوج باشد ($i \% 2 == 0$)، agent1 به عنوان بازیکن اول (مهره سیاه و شروع‌کننده) و agent2 به عنوان بازیکن دوم (مهره سفید) به تابع play_game فرستاده می‌شوند. امتیاز بازگردانده شده به ترتیب به score1 و score2 نسبت داده می‌شود.

```
else:
    score = play_game(agent2, agent1)
    # چون جای بازیکنها عوض شده، امتیازها رو هم برعکس می‌خوانیم
    score2 = score[0]
    score1 = score[1]
```

اگر شماره بازی فرد باشد، جای عامل‌ها عوض می‌شود تا agent2 شروع‌کننده باشد. از آنجا که خروجی play_game همیشه امتیازات را به ترتیب چیدمان ورودی‌ها می‌دهد، این بار اندیس صفر خروجی متعلق به عامل دوم (score2) و اندیس یک متعلق به عامل اول (score1) خواهد بود.

```
# چک می‌کنیم کی برده
if score1 > score2:
    agent1_wins += 1
elif score1 < score2:
    agent2_wins += 1
else:
    draws += 1
```

امتیاز نهایی عامل اول و دوم به صورت مستقل از رنگ مهره با هم مقایسه شده و بسته به نتیجه، به شمارنده برد عامل اول، برد عامل دوم یا تساوی‌ها یک واحد اضافه می‌شود.

```
# حساب کردن درصد برد عامل اول
win_rate = (agent1_wins / num_games) * 100

print(f"Total Games: {num_games}")
print(f"Agent 1 Wins: {agent1_wins}")
print(f"Agent 2 Wins: {agent2_wins}")
print(f"Draws: {draws}")
print(f"Agent 1 Win Rate: {win_rate}%")
print("-" * 40)

return win_rate
```

پس از اتمام همه بازی‌ها، فرمول درصد برد عامل اول را محاسبه کرده و اطلاعات آماری بازی را جمله تعداد کل، بردهای هر کدام، تعداد مساوی‌ها و نرخ نهایی برد عامل اول را روی صفحه چاپ می‌کنند.


```
if __name__ == "__main__":
    # ساختن عامل‌ها
    minimax_player = MinimaxAgent(depth=3)
    alphabeta_player = AlphaBetaAgent(depth=4)
    random_player = RandomAgent()
    greedy_player = GreedyAgent()
```

این بخش، نقطه ورود برنامه (نقطه شروع اجرا) است که در آن ۴ نوع عامل مختلف با مشخصات و عمق‌های تعیین شده ساخته می‌شوند.

```
print("Test 1: Minimax vs Random")
test_agents(minimax_player, random_player, num_games=20)

print("Test 2: AlphaBeta vs Random")
test_agents(alphabeta_player, random_player, num_games=20)

print("Test 3: Minimax vs Greedy")
test_agents(minimax_player, greedy_player, num_games=20)

print("Test 4: AlphaBeta vs Greedy")
test_agents(alphabeta_player, greedy_player, num_games=20)

print("Test 5: AlphaBeta vs Minimax (Head-to-Head)")
test_agents(alphabeta_player, minimax_player, num_games=20)
```

در نهایت پنج آزمایش کلیدی و ترکیبی (تقابل Minimax با رندوم، Alpha-Beta با رندوم، Minimax با حریصانه، Alpha-Beta با حریصانه و در نهایت رقابت مستقیم دو عامل هوشمند) برای ارزیابی جامع فراخوانی می‌شوند.

نتایج و مقایسه:

آزمایش‌ها در محیط 6×6 و طی ۲۰ بازی برای هر مقابله انجام شد. عامل Minimax با عمق ۳ و عامل Alpha-Beta با عمق ۴ تنظیم شدند (سرعت بالاتر Alpha-Beta امکان بررسی عمق بیشتر به ما داد). عملکرد هر کدام با agentهای متفاوت بررسی می‌کنیم:

در مقابل Random Agent: هر دو الگوریتم توانستند عامل تصادفی را با نرخ برد ۱۰۰٪ شکست دهند.

در مقابل Greedy Agent: عامل حریصانه که در لحظه به دنبال تصاحب بیشترین مهره است، در برابر استراتژی بلندمدت الگوریتم‌های ما ضعیف عمل کرده و نرخ برد عامل‌های ما حدود ۹۰ تا ۱۰۰٪ بود.

رقابت مستقیم دو الگوریتم (Alpha-Beta vs Minimax): عامل Alpha-Beta به دلیل جستجو در عمق ۴ (یک لایه عمیق‌تر از Minimax) توانست پیش‌بینی‌های بهتری انجام دهد و در اکثر بازی‌ها پیروز شود.

❖ جدول مقایسه Minimax و Alpha – Beta :

شماره تست	نام مسابقه Agent 1 vs) (Agent 2	تعداد بازی	وضعیت عمق جستجو	بردهای Agent1	بردهای Agent2	تعداد مساوی	درصد برد نهایی Agent1
۱	Minimax vs Random	۲۰	عمق = ۳	۱۹	۱	۰	۹۵.۰٪
۲	AlphaBeta vs Random	۲۰	عمق = ۴	۲۰	۰	۰	۱۰۰.۰٪
۳	Minimax vs Greedy	۲۰	عمق = ۳	۲۰	۰	۰	۱۰۰.۰٪
۴	AlphaBeta vs Greedy	۲۰	عمق = ۴	۲۰	۰	۰	۱۰۰.۰٪
۵	AlphaBeta vs Minimax (Head-to-Head)	۲۰	عمق ۴ VS (AB) عمق ۳ (MM)	۱۰	۱۰	۰	۵۰.۰٪

```

/usr/local/bin/python3 /Users/farnoush/Downloads/AI26-PRJ3/main.py
➔ AI26-PRJ3 /usr/local/bin/python3 /Users/farnoush/Downloads/AI26-PRJ3/main.py
Test 1: Minimax vs Random
Total Games: 20
Agent 1 Wins: 19
Agent 2 Wins: 1
Draws: 0
Agent 1 Win Rate: 95.0%
=====
Test 2: AlphaBeta vs Random
Total Games: 20
Agent 1 Wins: 20
Agent 2 Wins: 0
Draws: 0
Agent 1 Win Rate: 100.0%
=====
Test 3: Minimax vs Greedy
Total Games: 20
Agent 1 Wins: 20
Agent 2 Wins: 0
Draws: 0
Agent 1 Win Rate: 100.0%
=====
Test 4: AlphaBeta vs Greedy
Total Games: 20
Agent 1 Wins: 20
Agent 2 Wins: 0
Draws: 0
Agent 1 Win Rate: 100.0%
=====
Test 5: AlphaBeta vs Minimax (Head-to-Head)
Total Games: 20
Agent 1 Wins: 10
Agent 2 Wins: 10
Draws: 0
Agent 1 Win Rate: 50.0%
=====

```

❖ مقایسه پیچیدگی زمانی دو الگوریتم:

تعداد حرکات مجاز در هر مرحله را b و عمق درخت را d قرار می‌دهیم. از آنجایی که الگوریتم Minimax برای رسیدن به مقدار دقیق بهینه در هر گره، تمام شاخه‌ها را تا عمق d پیمایش می‌کند و گره‌های تولید شده را بررسی می‌کند؛ پیچیدگی زمانی آن همیشه ثابت و برابر $O(b^d)$ است. از طرفی کارایی الگوریتم Alpha-Beta ارتباط مستقیم با ترتیب بررسی حرکات دارد. اگر حرکات بهینه در ابتدای لیست جستجو قرار گیرند، مقدار کران‌های α و β سریع‌تر به‌روزرسانی می‌شود و شرایط هرس کردن (Pruning) زودتر فراهم می‌شود.

در نتیجه بخش‌های زیادی از درخت جستجو که اهمیت استراتژیک ندارند بررسی نمی‌شوند. در بهترین حالت (حرکات به خوبی مرتب شده‌اند) پیچیدگی زمانی این الگوریتم $O(b^{\frac{d}{2}})$ است.

کاهش یافتن و نصف شدن توان در پیچیدگی زمانی Alpha-Beta به این معناست که در یک بازه زمانی ثابت، اگر Minimax بتواند تا عمق ۳ نفوذ کند، الگوریتم Alpha-Beta می‌تواند تا عمق ۶ نفوذ کند بدون اینکه زمان بیشتری مصرف کند. این فرمول ریاضی، علت برتری عامل Alpha-Beta در رقابت مستقیم دو الگوریتم را توجیه می‌کند.

تحلیل نتایج و پاسخ به سوالات:

۱. افزایش عمق جستجو چه تاثیری بر کیفیت بازی داشت؟

افزایش عمق به agent اجازه می‌دهد تا پیامدهای حرکات خود را در آینده دورتر پیش‌بینی کند. این کار باعث می‌شود عامل در تله‌های Greedy agent نیفتد و استراتژی‌های بلندمدت (مثل فدا کردن مهره در کوتاه‌مدت برای گرفتن گوشه در بلندمدت) را با موفقیت اجرا کند. کیفیت بازی بالا می‌رود اما هزینه پردازشی آن نیز به صورت نمایی افزایش می‌یابد.

۲. الگوریتم Alpha-Beta چه تأثیری بر زمان اجرا داشت؟

الگوریتم Alpha-Beta با استفاده از تکنیک هرس کردن (Pruning) شاخه‌هایی که قطعاً توسط حریف یا خود ما انتخاب نمی‌شوند را حذف می‌کند. این کار بدون اینکه تاثیری بر کیفیت تصمیم‌گیری نهایی داشته باشد (همان خروجی Minimax را می‌دهد)، تعداد گره‌های بررسی شده را کاهش داده و در نتیجه برنامه در زمان کمتری اجرا می‌شود.

۳. چه معیارهایی را در تابع ارزیابی استفاده کردید؟

همان طور که در بخش ۳ توضیح داده شد، از سه معیار اصلی استفاده شد:

۱. توازن مهره‌ها (Coin Parity)

۲. تحرک پذیری یا میزان آزادی حرکت (Mobility)

۳. تصرف گوشه‌ها (Corners Occupancy)

۴. مهم ترین عامل موفقیت عامل شما چه بود؟

مهم ترین عامل موفقیت، اختصاص دادن وزن بسیار بالا (۱۵۰) به گوشه‌های صفحه در تابع ارزیابی بود. الگوریتم متوجه شد که گرفتن گوشه‌ها کلید پیروزی است و برای رسیدن به آن حاضر بود در کوتاه مدت تعداد مهره‌های کمتری داشته باشد. همچنین ترکیب این مورد با معیار Mobility باعث شد عامل حریف را مجبور کند تا حرکات بدی انجام دهد.

۵. نقاط ضعف تابع ارزیابی شما چیست؟

بزرگترین نقطه ضعف این تابع ارزیابی، ثابت بودن وزن‌های آن در تمام مراحل بازی است. در بازی اتلو، در اوایل بازی تعداد مهره‌ها چندان مهم نیست و قابلیت تحرک اهمیت بیشتری دارد، اما در اواخر بازی اختلاف تعداد مهره‌ها بسیار مهم می‌شود. تابع فعلی این تغییر اهمیت معیارهای بازی را درک نمی‌کند و از اول تا آخر بازی وزن ثابتی به ویژگی‌ها می‌دهد. همچنین در این تابع، خانه‌های خطرناک مجاور گوشه (X-squares و C-squares) به طور خاص جریمه نشده‌اند.

جمع‌بندی:

هدف اصلی این پروژه، پیاده‌سازی و مقایسه‌ی دو الگوریتم جستجوی هوشمند، یعنی Minimax و Alpha-Beta، در بازی اتلو بود. برای تصمیم‌گیری بهتر از یک تابع ارزیابی خطی استفاده شد که سه مؤلفه‌ی اصلی را در نظر می‌گرفت: توازن مهره‌ها، تحرک‌پذیری (میزان آزادی حرکت) و تصاحب گوشه‌های صفحه. از آنجا که گوشه‌ها در اتلو از اهمیت بالایی برخوردارند و نقش مهمی در برتری موقعیت دارند، بیشترین وزن به این بخش داده شد و ضریب ۱۵ برای آن در نظر گرفته شد.

از نظر عملکرد، هر دو عامل هوشمند به دلیل برخورداری از نگاه آینده‌نگری و ارزیابی چندمرحله‌ای، توانستند عامل‌های تصادفی و حریصانه را با نرخ بردی نزدیک به ۱۰۰ درصد شکست دهند. با این حال، در رقابت مستقیم میان این دو الگوریتم، Alpha-Beta عملکرد بهتری داشت. علت این برتری آن بود که این الگوریتم با استفاده از مکانیزم هرس آلفا-بتا و حذف شاخه‌های غیرضروری در شرط $\beta \leq \alpha$ ، زمان محاسباتی کمتری مصرف کرد و در نتیجه توانست جستجوی خود را یک لایه عمیق‌تر از Minimax ادامه دهد به‌طوری که عمق جستجوی آن به ۴ رسید، در حالی که Minimax تنها تا عمق ۳ پیش رفت. همین یک لایه نگاه عمیق‌تر به آینده، عامل تعیین‌کننده در پیروزی الگوریتم Alpha-Beta بود.